引用

本质：小指针（功能与指针常量一样，只能修饰一个指定的地址，不能更改）

普通用法

例子：

```
int a = 10;
int &b = a;
```

注意

```
int main()
{
    int a = 10;
    int b = 20;
    // int &c;     //错误，引用必须先初始化
    int &c = a;     //一旦初始化，就不能更改指向对象
    c = b;           //这是赋值操作，不是更改引用操作

    cout << "a = " << a << endl;    // a = 20
    cout << "b = " << b << endl;    // b = 20
    cout << "c = " << c << endl;    // c = 20

    system("pause");
    return 0;
}
```

引用与函数

```
int& test()
{
    static int a = 20;          //static关键词 防止栈区a的值被销毁
    return a;
}

int main()
{
    int& ref = test();
    cout << "ref = " << ref << endl;      //   20
    cout << "ref = " << ref << ebdl;      //   20        二次输出是验证a的值是否被销毁
```

```

    test() = 1000;                    //函数可作为左值，进行赋值操作

    cout << "ref = " << ref << endl;      //   1000
    cout << "ref = " << ref << endl;      //   1000

    system("pause");
    return 0;
}
```

函数细节

函数默认参数

在函数形参中是可以有默认值的

注意 1：在形参中设置默认值时，**当一个参数设置默认值后，在这个参数后面的所有形参都需要设置默认值。**例如下述代码err函数

注意 2：在声明函数且使用函数时，声明和使用只能有一个形参的初始值，具体见函数err2.

```
#include<iostream>
uisng namespace std;

int func(int a = 10, int b = 20, int c = 30)
{
    return a + b + c;
}

int err(int a, int b = 20, int c)
{
    return a + b + c;
}

int err2(int a = 10, int b = 20, int c = 30);   // 声明

int main()
{
    cout << func(100) << endl;
    //结果是150， 因为func只传入了一个参数，传给了 a ，当实参和形参都有初始值的话，优先实参传入的值

    cout << err() << endl;  //err函数是一个错误函数，原因在注意1中

    cout << err2() <<endl;  //err2函数是一个错误函数，原因在注意2中
    system("pause");
    return 0;
}

int err2 (int a = 10, int b = 20, int c = 30)    // 使用
{
    return a + b + c;
```

```
}
```

函数的占位参数

在函数形参中可以有占位参数，用来占位操作，调用函数时必须填补该位置， 但实参传入的值在占位函数中无效

注意:占位参数也是可以设置默认参数的

语法：返回类型 + 函数名（数据类型）

```
#include<iostream>
using namespace std;

void func1(int)
{
    cout << "test1" << endl;
}

void func2(int a, int 10)    //设置默认参数
{
    cout << "test2" << endl;
}

int main()
{

    func1(10);
    func2(10, 10);

    system("pause");
    return 0;
}
```

函数重载

作用：函数名可以相同，提高复用性

函数重载满足的条件：

- 同一个作用域下
- 函数名相同
- 函数参数 **类型不同** 或者 **个数不同** 或者 **顺序不同**

注意：

- 函数返回值不可以作为函数重载的条件(就是返回值不同但函数名相同不能进行函数重载)
- 引用作为重载条件

- 当使用函数重载又使用默认参数时，可能会出现**二义性**，会报错，详见 func2 函数，所以**在使用函数重载后就尽量不使用默认参数，避免出错**

```
#include<iostream>
using namespace std;

void func()
{
    cout << "func()的调用" << endl;
}

void func(int a)
{
    cout << "func(int a)的调用" << endl;
}

void func(double b)
{
    cout << "func(double b)的调用" << endl;
}

void func(int a, double b)
{
    cout << "func(int a, double b)的调用" << endl;
}

void func(double b, int a)
{
    cout << "func(doule b, int a)的调用" << endl;
}

int func()                      //错误，函数返回值不同不能作为函数重载的条件
{
    cout << "func()的调用" << endl;
}

void func(int &a)
{
    cout << "func(int &a)的调用" << endl;
}

void func(const int &a)
{
    cout << "func(const int &a)的调用" << endl;
}

void func2(int a, int b = 10)
{
    cout << "func(int a, int b = 10)的调用" << endl;
}

void func2(int a)
{
```

```
    cout << "func(int a)的调用" << endl;
}

int main()
{
    func();          //func()的调用
    func(10);         //func(int a)的调用
    func(3.14);         //func(double b)的调用
    func(10, 3.14);        //func(int a, double b)的调用

    int a = 10;
    func(a);               //func(int &a)的调用
    func(10);               //func(const int &a)的调用

    func2(a);           //报错，因为出现二义性，不知道调用func2中的哪一个函数

    system("pause");
    return;
}
```

类与对象

面向对象的三大特点：**封装、继承和多态**

封装

封装的意义

- 将属性和行为作为一个整体，表现生活中的事物
- 将属性和行为加以权限控制

意义一：在设计类的时候，属性和行为写在一起，表现事物

语法：class + 类名 { 属性/行为 };

```
#include <iostream>
#include <string>
using namespace std;

class Student
{
    public:
    string SName;
    int SId;

    public:
    void setName(string name)
    {
        SName = name;
    }

```

```
    void setId(int id)
    {
        SId = id;
    }

    void showInf()
    {
        cout << "姓名:" << SName << "，学号:" << SId << endl;
    }
};

int main()
{
    Student stu1;
    stu1.SName = "张三";
    stu1.SId = 20;
    stu1.showInf();     // 姓名：张三，学号：20

    Student stu2;
    stu2.setName("李四");
    stu2.setId(2);
    stu2.showInf();     // 姓名：李四，学号：2

    system("pause");
    return 0;
}

```

意义二：类在设计时，可以把属性和行为放在不同的权限下，加以控制

三种权限：

- public　　公共权限（**成员在类内可以访问，类外也可以访问**）
- protected　保护权限（**成员在类内可以访问，类外不能访问，子类可以访问父类的protected内容**）
- private　　私有权限（**成员在类内可以访问，类外不能访问，子类不能访问父类的private内容**）

```
#include<iostream>
#include<string>
using namespace std;

class Person
{
    public:
        string name;
    protected:
        string car;
    private:
        int password;

    public:
    void set_Inf()
    {
```

```
        name = "张三";
        car = "大众";
        password = 123;
    }
};

int main()
{
    Person p;

    p.set_Inf();
    p.name = "李四";
    p.car = "宝马";        //error:类外不可以访问protected权限
    p.password = 456;     //error: 类外不可以访问private权限

    system("pause");
    return 0;
}
```

class与struct区别

区别:

- struct内默认访问权限是public
- class内默认访问权限是private

```
struct A
{
    int a;
};

class B
{
    int b;
};

int main()
{
    A stru;
    B cla;

    stru.a = 1;
    cla.b = 1;       //error:class类中成员默认private权限

    return 0;
}
```

成员属性的只读、只写

将成员属性先设置为私有，即使用class进行封装，而不用struct

```
class Person
{
    //设置姓名可读可写
    //年龄只能读
    //号码只能写
    string name;
    int age = 18;
    string num;

    //可读可写
    void setName(b_name)
    {
        name = b_name;
    }

    string getName()
    {
        return name;
    }

    // 只读
    int getAge()
    {
        return age;
    }

    //只写
    void setNum(string n)
    {
        num = n;
    }
};

int main()
{
    Person p;

    p.name = "张三";  // 不可访问

    p.setName("张三");
    cout << "姓名为："<< p.getName() << endl;

    cout << "姓名为："<< p.getAge() << endl;

    p.setNum("1234");

    return 0;
}
```

对象的初始化和清理

构造函数和析构函数

构造函数：创建对象自动调用，一般为成员属性赋初始值

析构函数：对象销毁前自动调用，一般执行一些清理工作

```
class Person
{
    //构造函数，只调用一次
    //可以有参数，可以发生重载
    Person()
    {
        cout << "Person 的构造函数" << endl;
    }

    //析构函数，只调用一次
    //不能有参数，不可以发生重载
    ~Person()
    {
        cout << "Person 的析构函数" << endl;
    }
};

// 也可以在类外写构造函数(其他类中成员也可以这样书写)
class House
{
    public:
    House();
    string room;
};

House::House()
{
    room = "卧室";
    cout << "类外的构造函数" << endl;
}

void test()
{
    Person p;
    House house;
}

int main()
{
    test();

    return 0;
}

```

构造方法分类及调用

分类：无参构造、有参构造、拷贝构造

默认情况下，编译器至少给一个类添加3个上述构造函数

调用规则：

- 如果用户定义了有参构造函数，那么编译器不再提供默认无参构造，但是会提供拷贝构造函数
- 如果用户定义了拷贝构造函数， 编译器就不会再提供其他构造函数

调用方法：括号法、显式法、隐式转换法

注意：

- 调用无参构造函数不能加括号，如果加了编译器会认为这是一个函数声明
- 不能利用 拷贝构造函数 初始化匿名对象 编译器会认为是对象声明

```
class Person
{
public:
    int age;

    Person()
    {
        cout << "无参构造" << endl;
    }

    Person(int a)
    {
        age = a;
        cout << "有参构造" << endl;
    }

    Person(const Person &p)
    {
        age = p.age;
        cout << "拷贝构造函数" << endl;
    }
};

void test()
{
    // 括号法
    Person p1;
    Person p2(10);
    Person p3(p2);
    //Person p4()    error: 会认为是一个函数的声明 例如 void add（）

    //显式发
    Person p4;
    Person p5 = Person(10);
    Person p6 = Person(p5);

```

```
    //只写右边表示匿名对象，当前行被执行后立即被释放
    Person(50);
    // Person(p6)     error: Person(p6) = Person p6  会被认为一个对象声明

    //隐式转换法
    Person p7 = 10;
    Person p8 = p7;

}

int main()
{
    test();
    return 0;
}
```

拷贝构造函数调用时机

- 使用一个已经创建完毕的对象来初始化一个新对象
- 值传递的方式给函数参数传值
- 以值传递返回局部对象

```
class Person
{
public:
    int age;

    Person()
    {
        cout << "无参构造" << endl;
    }

    Person(int a)
    {
        age = a;
        cout << "有参构造" << endl;
    }

    Person(const Person &p)
    {
        age = p.age;
        cout << "拷贝构造函数" << endl;
    }
};

void test01()
{
    Person p1;
    Person p2(p1);
}

void doWork01(Person p)
```

```
{

}

void test02()
{
    Person p3;
    doWork(p3);    // 此处为传值，不是传地址，因此只是将对象拷贝一个副本传入上述函数，因此传入时需用拷贝构造复制一份对象
}

void doWork02()
{
    Person p4;
    return p4;    //此处返回的是值表示地址，因此只是将对象拷贝一个副本返回，因此返回时需用拷贝构造复制一份对象
}

void test03()
{
    Person p5 = doWork02();
}

int main()
{
    test01();
    test02();
    test03();
    return 0;
}
```

深拷贝与浅拷贝

浅拷贝：简单的赋值操作

深拷贝：在堆区内重新开辟内存，进行拷贝工作

注意：如果有成员属性在堆区开辟内存，那么就需要之间提供一个拷贝构造函数，防止浅拷贝带来的重复释放问题

```
class Person
{
    public:
    int m_age;
    int *m_height;

    Person()
    {
        cout << "无参构造" << endl;
    }

    Person(int age, int height)
    {
        cout << "有参构造" << endl;
```

```

        m_age = age;
        m_hight = new int(height);
    }

    // 如果使用
    Person(const Person & p)
    {
        cout << "拷贝构造函数" << endl;
        m_age = p.m_age;
        //m_height = p.m_height     编译器提供的默认拷贝构造函数内容
        m_height = new int(*p.height);
    }

    ~Person()
    {
        cout << "析构函数" << endl;
        if(m_height != NULL)
        {
            delete m_height;
            m_height = NULL;
        }
    }
};

void test()
{
    Person p1(18,180);
    Person p2(p1);

    cout << "p1的年龄：" << p1.m_age << " 身高：" << *p1.m_height << endl;
    cout << "p2的年龄：" << p2.m_age << " 身高：" << *p2.m_height << endl;
}

int main()
{
    test();
    return 0;
}
```

初始化列表赋初始值

```
class Person
{
    private:
    int m_A, m_B, m_C;

    public:
    //传统赋初值
    Person(int a, int b, int C)
    {
        m_A = a;
        m_B = b;
```

```
        m_C = c;
    }

    //初始化列表方式初始化
    Person(int a, int b, int c): m_A(a), m_B(b), m_C(c)
    {}

};

int main()
{
    Person(1,2,3);
    return 0;
}
```

类对象作为类成员

当其他类对象作为本类成员，构造时先构造类对象，再构造自身，析构的顺与构造相反

```
class Phone
{
    public:
    int phoneNum;
    Phone(int num)
    {
        cout << "这是Phone的构造函数" << endl;
        phoneNum = num;
    }

    ~Phone()
    {
        cout << "这是Phone的析构函数" << endl;
    }
};

class Person
{
    public:

    string pName;
    Phone ph;

    Person(string name, int num):ph(num)
    {
        cout << "这是Person的构造函数" << endl;
        pName = name;
    }

    ~Person()
    {
        cout << "这是Person的析构函数" << endl;
    }
};
```

```

void test()
{
    Person p("张三",123);
    cout << p.pName << "的手机号为" << p.ph.phoneNum << endl;


    //输出结果:
    //这是Phone的构造函数
    //这是Person的构造函数
    //张三的手机号为123
    //这是Person的析构函数
    //这是Phone的析构函数
}

int main()
{
    test();
    return 0;
}
```

静态成员

- 静态成员变量
  - 可以通过类名进行访问
  - 所以对象共享同一份数据
  - 在编译阶段分配内存
  - **类内声明，类外初始化**

```
#include<string>

class Person
{
    public:
    static int num;

    private:
    static int m_num;
};

int Person::num = 100;
int Person::m_num = 100;

void test()
{
    //通过对象访问
    Person p;
    cout << p.num << endl;
    //100

    Person p2;
```

```
    p2.num = 200;
    cout << p.num << endl;
    //200

    //通过类名访问
    cout << Person::num << endl;
    //cout << Person::m_num << endl;   error:私人权限，类外无法访问
}

int main()
{
    test();
    return 0;
}
```

- 静态成员函数
  - 所有对象共享同一个函数
  - 静态成员函数只能访问静态成员变量

```
class Person
{
    public:
    static void func1()
    {
        cout << "func1的调用" << endl;
        num01 = 1;
        //num02 = 2;   静态函数不能访问非静态成员变量
    }

    static int num01;
    int num02;

    private:
    static void func2()
    {
        cout << "func2的调用" << endl;
    }
};

void test()
{
    // 通过对象访问
    Person p;
    cout << p.func1 << endl;

    //通过类名访问
    cout << Person::func1 << endl;

    //cout << Person::func2 << endl;   error:类外不能访问私有权限函数
}

```

```
int main()
{
    test();
    return 0;
}

```

对象模型和this指针

类对象的大小

成员变量和成员函数是分开存储的

```
class Test
{

};

class Person
{
    int a;      // 非静态成员变量，属于类的对象上，sizeof会分配大小

    static int b;   // 静态成员变量，不属于类对象上

    void func(){}   // 非静态成员函数，不属于类对象上

    static void func2(){}   //静态成员函数，不属于类的对象上
};

void test()
{
    Test t;
    cout << "size of t = " << sizeof(t) << endl;
    //1     空对象占用内存空间为1，为了区分不同空对象所占内存的位置

    Person p;
    cout << "size of p = "  << sizeof(p) << endl;
    //4
}

int main()
{
    test();
    return 0;
}
```

this指针

this指针的作用

- 当名称相同时，解决名称冲突
- 返回对象本身，用*this

注意：this指针不能修改指针的指向(this指针全写定义是 **类名+* const this**)

```
class Person
{
    public:
    int age;

    Person(int age)
    {
        this->age = age;
        //this = NULL;      error:this指针不能修改指针的指向
    }

    Person& age_Add(Person &p)
    {
        this->age += p.age;
        return *this;
    }
};

//解决名称冲突
void test01()
{
    Person p(18);
    cout << "年龄为" << p.age << endl;
    //18
}

//返回对象本身（链式思想）
void test02()
{
    Person p1(10);
    Person p2(10);

    p2.age_Add(p1).age_Add(p1).age_Add(p1);
    cout << "p2年龄为" << p2.age << endl;
    //40
}

int main()
{
    test01();
    test02();
    return 0;
}
```

空指针访问成员函数

空指针可以调用不访问成员变量的成员函数

```
class Person
{
    public:
    int age ;

    void showTest()
    {
        cout << "这是一个没有成员变量的成员函数" << endl;
    }

    void showAge()
    {
        cout << "年龄为" << age << endl;
    }
};

void test01()
{
    Person *p = NULL;
    p -> showTest();
    p -> showAge();     //error:传入空指针无法调用成员变量
}

int main()
{
    test01();
    return 0;
}
```

const 修饰类相关

常函数：成员函数后加const后我们称为这个函数为常函数（成员函数后面的`const`关键字并不是用来修饰返回类型的，而是用来修饰成员函数本身的）

- 常函数内不可以修改成员属性
- 成员属性声明时加关键字mutable后，在常函数中依然可以修改（常对象中也可以修改）

常对象：声明对象前加const

- 常对象只能调用常函数

```
class Person
{
    public:
    int age;
    mutable int num;

    void classTest() const
    {
```

```
        //age = 18; //error: 常函数不能修改成员变量的值
        num = 100;
    }

    void func()
    {}


};

void test()
{
    Person p;
    p.classTest();

    //常对象
    const Person p1;
    p.classTest();
    //p.func();     error: 常对象只能调用常函数
}

int main()
{
    test();
    return 0;
}
```

友元

可以访问私有属性，关键词**friend**

全局函数作为友元

在类中声明友元

```
class Building
{
    friend void good_Gay(BUilding &building);

    public:
    string sittingRoom;

    private:
    string bedRoom;

    public:
    Building()
    {
        sittingRoom = "客厅";
        bedRoom = "卧室";
    }
```

```
};

void good_Gay(BUilding &building)
{
    cout << "全局函数 正在访问:" << building -> sittingRoom << endl;
    cout << "全局函数 正在访问:" << building -> bedRoom << endl;
}

void test01()
{
    Building building;
    good_Gay(&building);
}

int main()
{
    test01();
    return 0;
}
```

类作友元

在类中声明友元

```
class Building
{
    friend class GoodGay;
    public:
    string sittingRoom;

    Building()
    {
        sittingRoom = "客厅";
        bedRoom = "卧室";
    }

    private:
    string bedRoom;
};

class GoodGay
{
    public:
    Building *building;

    GoodGay()
    {
        building = new Building;
    }

    void visitRoom()
    {
        cout << "正在访问:" << building -> sittingRoom << endl;
```

```
        cout << "正在访问:" << building -> bedRoom << endl;
    }
};

void test()
{
    GoodGay one;
    one.visitRoom();
}

int main()
{
    test();
    return 0;
}
```

成员函数作为友元

需要声明是哪个类中的成员函数

注意：下列编译必须在类外定义成员函数的内容，因为不管是哪个类在前都会报错显示使用了为定义的类型，这是因为都是在编译时按顺序运行，因此不能在某个类成员定义之前就使用

```
class Building;

class GoodGay
{
    public:
    Building *building;

    GoodGay();

    void visitRoom();
};

class Building
{
    friend void GoodGay::visitRoom();
    public:
    string sittingRoom;

    Building()
    {
        sittingRoom = "客厅";
        bedRoom = "卧室";
    }

    private:
    string bedRoom;
};

GoodGay::GoodGay()
{
```

```
    building = new Building;
}

void GoodGay::visitRoom()
{
    cout << "正在访问:" << building -> sittingRoom << endl;
    cout << "正在访问:" << building -> bedRoom << endl;
}

void test()
{
    GoodGay one;
    one.visitRoom();
}

int main()
{
    test();
    return 0;
}
```

运算符重载

对已有的运算符重新进行定义，赋予其另一种功能，以适应不同的数据类型

关键词：operator + 符号

加号的运算符重载

实现对两个自定义数据类型的相加（运算符重载也可以发生函数重载）

- 通过成员函数进行重载
- 通过全局函数进行重载

注意：

- 对于内置的数据类型的表达式的运算符是不可能改变的（例如1+1=2）
- 不要滥用运算符重载

```
class Person
{
    public:

    int mA;
    int mB;

    //成员函数进行重载
    /*Person operator + (Person &p)
    {
        Person temp;
        temp.mA = this->mA + p.mA;
        temp.mB = this->mB + p.mB;
        return temp;
```

```
    }*/
};

Person operator + (Person &p1, Person &p2)
{
    Person temp;
    temp.mA = p1.mA + p2.mA;
    temp.mB = p1.mB + p2.mB;
    return temp;
}

Person operator + (Person &p1, int num)
{
    Person temp;
    temp.mA = p1.mA + num;
    temp.mB = p1.mB + num;
    return temp;
}

void test()
{
    Person p1;
    p1.mA = 10;
    p1.mB = 10;
    Person p2;
    p2.mA = 10;
    p2.mB = 10;

    Person p3 = p1 + p2;

    //成员函数调用本质
    //Person p3 = p1.operator+(p2);

    //全局函数重载本质调用
    //Person p3 = operator+(p1,p2);

    cout << "p3.mA = " << p3.mA << endl;
    cout << "p3.mB = " << p3.mB << endl;

    Person p4 = p1 + 100;

    cout << "p3.mA = " << p3.mA << endl;
    cout << "p3.mB = " << p3.mB << endl;
}

int main()
{
    test();
    return 0;
}
```

左移运算符重载

一般不会用成员函数重载左移运算符

```
class Person
{
    public:
    int mA, mB;

};

ostream& operator<<(ostream& cout, Person &p)
{
    cout << "mA =" << p.mA << ",mB = " << p.mB;
    return cout;
}

void test()
{
    Person p;
    p.mA = 10;
    p.mB = 10;

    cout << p << endl;
}

int main()
{
    test();
    return 0;
}
```

递增运算符重载

写为成员函数较为方便

注意：在重载左移运算符 `<<` 时，且又使用重载递增运算符 `++` 时，重载左移运算符的形参中**需要添加const关键字**，这样可以使 `p++` 生成的临时对象能够被正确传递和处理。

```
class Person
{
    public:
    int age;

    Person()
    {
        age = 18;
    }

    Person& operator++()
    {
        age++;
        return *this;
```

```
    }

    Person operator++(int)
    {
        Person temp = *this;
        age++;
        return temp;
    }

};

ostream& operator<<(ostream& cout, const Person &p)
{
    cout << p.age;
    return cout;
}

void test()
{
    Person p;

    cout << p++ << endl;
    //18

    cout << ++p << endl;
    //20
}

int main()
{
    test();
    return 0;
}
```

赋值运算符重载

赋值运算符 `operator=`，对属性进行值拷贝

```
class Person
{
    public:
    int *age;

    Person(int a)
    {
        age = new int(a);
    }

    ~Person()
    {
        if(age != NULL)
        {
            delete age;
```

```
            age = NULL;
        }
    }

    Person& operator=(Person &p)
    {
        if(age != NULL)
        {
            delete age;
            age = NULL;
        }

        age = new int(*p.age);

        return *this;
    }
};

void test()
{
    Person p1(18);
    Person p2(20);
    Person p3(30);

    p1 = p2 = p3;

    cout << "p1的年龄为: " << *p1.age << endl;
    cout << "p2的年龄为: " << *p2.age << endl;
    cout << "p3的年龄为: " << *p3.age << endl;
}

int main()
{
    test();
    return 0;
}
```

关系运算符重载

```
class  Person
{
    public:
    string name;
    int age;
    Person(string mName, int mAge)
    {
        name = mName;
        age = mAge;
    }

    bool operator==(Person &p)
    {
        if(this->name == p.name)
```

```
            return true;
        else
            return false;
    }

    bool operator!=(Person &p)
    {
        if(this->name == p.name)
            return false;
        else
            return true;
    }
};

void test()
{
    Person p1("孙悟空", 500);
    Person p2("孙悟空", 500);

    if(a == b)
    {
        cout << "a和b相等" << endl;
    }

    if(a != b)
    {
        cout << "a和b不相等" << endl;
    }
}

int main()
{
    test();
    return 0;
}
```

函数调用运算符重载

- 函数调用运算符（）也可以重载
- 由于重载后使用的方式非常像函数的调用，因此称为仿函数
- 仿函数没有固定写法，非常灵活

```
class Myprint
{
    public:
    void operator()(string text)
    {
        cout << text << endl;
    }
}

class MyAdd
```

```
{
    public:
    int operator()(int num1, int num2)
    {
        return num1 + num2;
    }
}

void test01()
{
    MyPrint myprint;
    myprint("hello world");
}

void test02()
{
    MyAdd myadd;
    int c = myadd(100, 100);
    cout << c << endl;

    //匿名对象调用
    cout << MyAdd(100, 100) << endl;
}

int main()
{
    test01();
    test02();

    return 0;
}
```

继承

继承的优点：减少代码量

基本语法

`class 子类 : 继承方式 父类名`

子类也称为派生类

父类也称为基类

子类中成员包含两大部分：

- 一类是从基类继承过来的，一类是自己增加的成员
- 从基类继承过来的表现其共性，而新增的成员体现了其个性

```
class BasePage
{
    public:
    void header()
    {
```

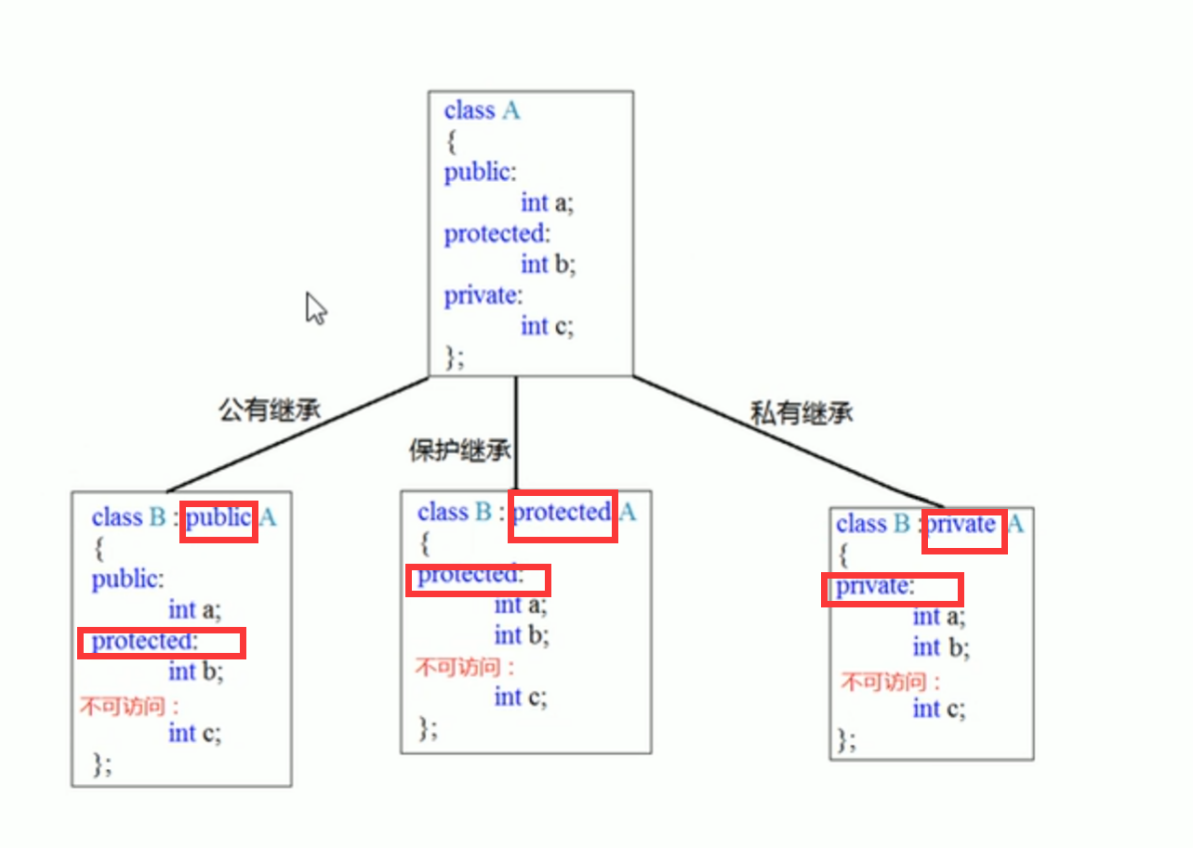
```
        cout << "首页...(公共头部)" << endl;
    }

    void footer()
    {
        cout << "帮助中心...（公共底部）" << endl;
    }

    void left()
    {
        cout << "Java、Python、C++..（公共分类列表）" << endl;
    }
}

//Java页面
class Java : public BasePage
{
    pubilc:
    void content()
    {
        cout << "Java学科视频" << endl;
    }
}

//C++页面
class Cpp : public BasePage
{
    pubilc:
    void content()
    {
        cout << "C++学科视频" << endl;
    }
}

//Java页面
class Python : public BasePage
{
    pubilc:
    void content()
    {
        cout << "Python学科视频" << endl;
    }
}

void test()
{
    cout << "Java下载视频页面如下 ： " << endl;
    Java ja;
    ja.header();
    ja.footer();
    ja.left();
    ja.content();

    cout << "C++下载视频页面如下 ： " << endl;
    Cpp cpp;
    cpp.header();
```

```
    cpp.footer();
    cpp.left();
    cpp.content();

    cout << "Python下载视频页面如下 : " << endl;
    Python python;
    python.header();
    python.footer();
    python.left();
    python.content();
}

int main()
{
    test();
    return 0;
}
```

继承方式

继承方式有三种:

- 公共继承
- 保护继承
- 私有继承




```
class Base1
{
public:
```

```
    int mA;

protected:
    int mB;

private:
    int mC;
};

class Son1 : public Base1
{
public:

    void func()
    {
        mA = 10;    //父类中的公共权限成员，到子类中依然是公共权限
        mB = 10;    //父类中的保护权限成员，到子类中依然是保护权限
        //mC = 10   error:父类中的私有权限成员，子类访问不到
    }
};

class Son2 : protected Base1
{
public:
    void func()
    {
        mA = 100;   //父类中公共成员 到子类中变为 保护成员
        mB = 100;
        //mC = 100; error:父类中的私有权限成员，子类访问不到
    }
};

class Son3 : private Base1
{
public:
    void func()
    {
        mA = 100;   //父类中公共成员 到子类中变为 私有成员
        mB = 100;   //父类中保护成员 到子类中变为 私有成员
        //mC = 100; error:父类中的私有权限成员，子类访问不到
    }
};

class GrandSon3 : public Son3
{
public:
    void func()
    {
        //mA = 100;     父类中已经变为私有成员
        //mB = 100;     父类中已经变为私有成员
        //mC = 100;     父类中已经变为私有成员
    }
};

void test01()
{
```

```
    Son1 s1;
    s1.mA = 100;
    //s1.mB = 100;  error:在Son1中mB是保护权限，类外不能访问

    Son2 s2;
    //s2.mA = 100   error:s2中mA已经变为保护属性

    Son3 s3;
    //s3.mA = 100   error:s3中mA已经变为私有属性
}

int main()
{
    test01();
    return 0;
}
```

继承中对象模型

父类中所有非静态成员属性都会被子类继承

父类私有成员属性是被编译器给隐藏了，因此是访问不到，但确实被继承下去了

```
class Base
{
    public:
    int mA;
    protected:
    int mB;
    private:
    int mC;
};

class Son : public Base
{
    public;
    int mD;
};

void test()
{
    cout << "size of Son = " << sizeof(Son) << endl;
}

int main()
{
    test();
    return 0;
}
```

继承中构造和析构顺序

子类继承父类后，当创建子类对象，也会调用父类的构造函数

继承中的构造和析构顺序如下：

先构造父类，再构造子类，析构的顺序与构造的顺序相反

```
class Base
{
    public:
    Base()
    {
        cout << "Base构造函数" << endl;
    }

    ~Base()
    {
        cout << "Base析构函数" << endl;
    }
};

class Son : public Base
{
    public:
    Son()
    {
        cout << "Son构造函数" << endl;
    }

    ~Son()
    {
        cout << "Son析构函数" << endl;
    }
};

void test()
{
    Son son;
    //结果：  Base构造函数
    //       Son构造函数
    //       Son析构函数
    //       Base析构函数
}

int main()
{
    test();
    return 0;
}
```

继承同名成员处理方式

- 访问子类同名成员 直接访问即可
- 访问父类同名成员 需要加作用域

注意：如果子类中出现和父类同名的成员函数，子类的同名成员会隐藏父类中所有同名成员函数，**如果想访问到父类中被隐藏的同名成员函数，需要加作用域**

```
class Base
{
    public:
    int mA;
    Base()
    {
        mA = 100;
    }

    void func()
    {
        cout << "Base-func 函数" << endl;
    }

    void func(int a)
    {
        cout << "Base-func(int a) 函数" << endl;
    }
};

class Son : public Base
{
    public:
    int mA;

    Son()
    {
        mA = 200;
    }

    void func()
    {
        cout << "Son-func 函数" << endl;
    }
};

void test()
{
    Son s;
    cout << "Son 下 mA = " << s.mA << endl;
    //200
    cout << "Base 下 mA = " << s.Base::mA << endl;
    //100

    s.func();
    // Son-func 函数
    s.Base::func();
```

```
    //Base-func 函数

    //s.func(100)   error:如果子类中出现和父类同名的成员函数，子类的同名成员会隐藏父类中所有同名成员函数，如果想访问到父类中被隐藏的同名成员函数，需要加作用域
    s.Base::func(100);
    //Base-func(int a) 函数
}

int main()
{
    test();
    return 0;
}
```

继承中同名静态成员处理方式

静态成员和非静态成员出现同名，处理方式一致，只是静态处理有两种方式

- 访问子类同名成员 直接访问即可
- 访问父类同名成员 需要加作用域

```
class Base
{
    public:
    static int mA;

    static void func()
    {
        cout << "Base-static func 函数" << endl;
    }

    static void func(int a)
    {
        cout << "Base-static func(int a) 函数" << endl;
    }
};
int Base::mA = 100;

class Son : public Base
{
    public:
    static int mA;

    static void func()
    {
        cout << "Son-static func 函数" << endl;
    }
};
int Son::mA = 200;

void test()
{
    Son s;
```

```
    //通过对象访问成员变量
    cout << "通过对象访问 " << endl;
    cout << "Son 下 mA = " << s.mA << endl;
    //200
    cout << "Base 下 mA = " << s.Base::mA << endl;
    //100

    //通过类名访问成员变量
    cout << "通过类名访问 " << endl;
    cout << "Son 下 mA = " << Son::mA << endl;
    //200
    cout << "Base 下 mA = " << Son::Base::mA << endl;
    //100


    //通过对象访问成员函数
    s.func();
    // Son-static func 函数
    s.Base::func();
    //Base-static func 函数

    //s.func(100)   error:如果子类中出现和父类同名的成员函数，子类的同名成员会隐藏父类中所有同名成员函数，如果想访问到父类中被隐藏的同名成员函数，需要加作用域
    s.Base::func(100);
    //Base-static func(int a) 函数

    //通过类名访问成员
    Son::func();
    // Son-static func 函数
    Son::Base::func();
    //Base-static func 函数
    Son::Base::func(100);
    //Base-static func(int a) 函数
}

int main()
{
    test();
    return 0;
}
```

多继承语法

C++实际开发中不建议用多继承

c++中允许一个类继承多个类

语法：`class 子类 : 继承方式 父类1， 继承方式 父类2...`

多继承可能会引发父类中有同名成员出现，需要加作用域区分

```
class Base1
{
    public:
```

```
    int mA;
    int mC;

    Base1()
    {
        mA = 100;
        mC = 100;
    }
};

class Base2
{
    public :
    int mB;
    int mC;

    Base2()
    {
        mB = 200;
        mC = 200;
    }
};

class Son : public Base1, public Base2
{
    public:
    int mD;

    Son()
    {
        mD = 1000;
    }
};

void test()
{
    Son s;
    cout << "size of Son = " << sizeof(s) << endl;
    //16;

    cout << "mA = " << s.mA << endl;
    cout << "mB = " << s.mB << endl;

    //父类同名
    cout << "Base1::mC" << s.Base1::mA << endl;
    cout << "Base2::mC" << s.Base2::mC << endl;
}

int main()
{
    test();
    return 0;
}
```

菱形继承

菱形继承概念：

两个子类继承同一个父类

又有某个类同时继承着两个子类

这种继承被称为菱形继承，或者钻石继承

注意：

- 菱形继承会造成二义性，当两个父类拥有相同数据，需要加以作用域区分
- 菱形继承会导致相同数据类型会继承两份，造成资源浪费
    - 利用虚继承，解决问题（在继承之前 加上关键字 virtual 变为虚继承）

      虚继承会将多个类继承的同一个成员，只保留一个类中的成员，其他类继承的该成员变为一个虚指针，指向该成员

```
class Animal
{
    public:
    int mAge;
};

class Sheep : virtual public Animal
{};

class Tuo : virtual public Animal
{};

class SheepTuo : public Sheep, public Tuo
{};

void test()
{
    SheepTuo st;
    //st.mAge = 18;      使用虚继承之前error:不知道访问哪一个

    st.Sheep::mAge = 18;
    st.Tuo::mAge = 28;

    cout << "Sheep::mAge = " << st.Sheep::mAge << endl;
    cout << "Tuo::mAge = " << st.Tuo::mAge << endl;
    // 虚继承前：18  28
    // 虚继承后：28  28

    //虚继承后可直接访问
    cout << "mAge = " << st.mAge << endl;
    //28
}

int main()
{
```

```
    test();
    return 0;
}
```

多态

多态的基本概念

多态分为两类

- 静态多态：函数重载和运算符重载属于静态多态，复用函数名
- 动态多态：派生类和虚函数实现运行时多态

静态多态和动态多态区别

- 静态多态的函数地址早绑定 — 编译阶段确定函数地址
- 动态多态的函数地址晚绑定 — 运行阶段确定函数地址

多态满足条件

- 有继承关系
- 子类重写父类中的虚函数

多态使用条件

- **父类指针或引用指向子类对象**

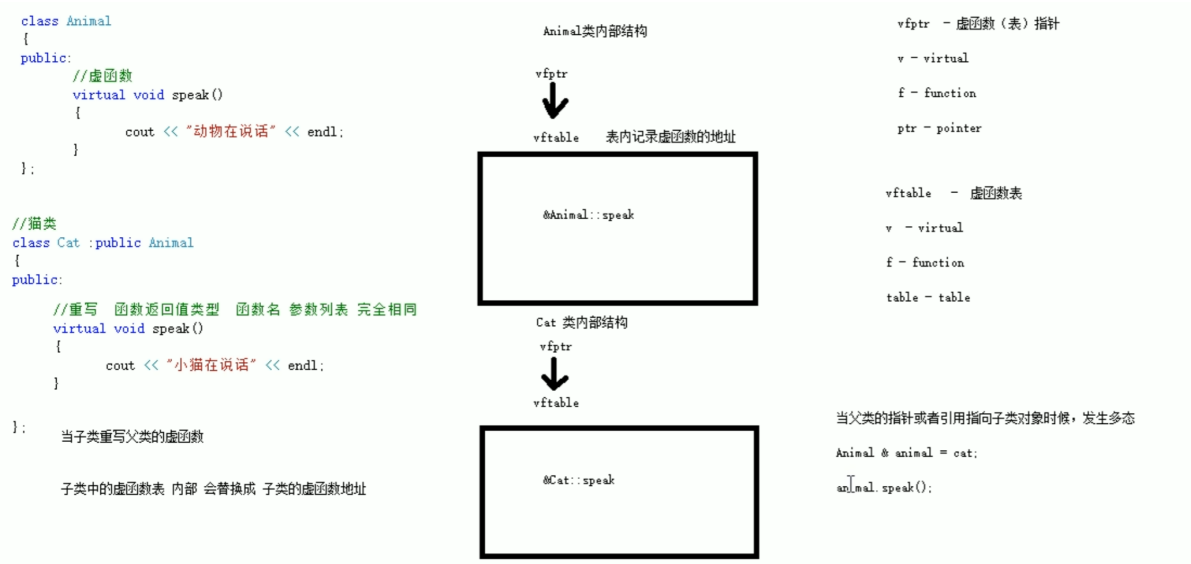
重写：函数返回值类 函数名 参数列表 完全一致称为重写

重载：参数列表的顺序，类型可能不一样，其他完全一致

```
class Animal
{
    public:
    virtual void speak()
    {
        cout << "动物在说话" << endl;
    }
};

class Cat : public Animal
{
    public:
    void speak()
    {
        cout << "小猫在说话" << endl;
    }
};

class Dog : public Animal
{
    public:
    void speak()
    {
```

```
        cout << "小狗在说话" << endl;
    }
};

void DoSpeak(Animal &animal)
{
    animal.speak();
}

void test()
{
    //在父类函数中加virtual前，因为都是静态多态，属于地址早绑定，在编译时就绑定
    //下面打印的结果都为：动物在说话
    Cat cat;
    DoSpeak(cat);

    Dog dog;
    DoSpeak(dog);
    //加入virtual后，属于动态多态，函数地址晚绑定 — 运行阶段确定函数地址
    //因此打印结果就为
    //小猫在说话
    //小狗在说话
}

int main()
{
    test();
    return 0;
}
```

内部逻辑图（关键：每一个表都有一个虚指针指向）



纯虚函数和抽象类

在多态中，通常父类中虚函数的实现是毫无意义的，主要都是调用子类重写的内容

因此可以将虚函数改为**纯虚函数**

纯虚函数语法：`virtual 返回值类型 函数名（参数列表） = 0;`

当类中有了纯虚函数，这个类也称为**抽象类**

抽象类特点：

- 无法实例化对象
- 子类必须重写抽象类的纯虚函数，否则也属于抽象类

```
class Base
{
    public:
    virtual void func() = 0;
};

class Son : public Base
{
public:
    virtual void func()
    {
        cout << "func函数调用" << endl;
    }
};

void test()
{
    //Base b;       error:抽象类无法实例化对象
    //new Base      error:抽象类无法实例化对象

    Base *base = new Son;
    base->func();
}

int main()
{
    test();
    return 0;
}
```

虚析构和纯虚析构

多态使用时，如果子类中有属性开辟到堆区，那么父类指针在释放时无法调用到子类的析构代码

解决方式：将父类中的析构函数改为**虚析构**或者**纯虚析构**

虚析构和纯虚析构共性：

- 可以解决父类指针释放子类对象
- 都需要有具体的函数实现

虚析构和纯虚析构区别：

- 如果是纯虚析构，该类属于抽象类，无法实例化对象

虚析构语法：

`virtual ~类名(){}`

纯虚析构语法：

`virtual ~类名() = 0;`

`类名::~类名(){}`

注意：

- 如果子类中没有堆区数据，可以不写为虚析构或纯虚析构
- 拥有纯虚析构函数的类也属于抽象类

```
class Animal
{
    public:
    Animal()
    {
        cout << "Animal 构造函数调用！" << endl;
    }
    virtual void Speak() = 0;

    //虚析构函数
    //virtual ~Animal
    //{
    //      cout << "Animal虚析构函数调用！" << endl;
    //}

    //纯虚析构函数，还需要类外实现，不然无法会报错，无法解析的外部命令
    virtual ~Animal() = 0;
}

Animal::~Animal()
{
```

```
    cout << "Animal 纯析构函数调用!" << endl;
}

class Cat : public Animal
{
    public:
    string *m_Name;

    Cat(string name)
    {
        cout << "Cat构造函数调用！" << endl;
        m_Name = new string(name);
    }

    virtual void Speak()
    {
        cout << *m_Name << "小猫在说话！" << endl;
    }

    ~Cat()
    {
        cout << "Cat析构函数调用!" << endl;
        if(this->m_Name != NULL)
        {
            delete m_Name;
            m_Name = NULL;
        }
    }
}

void test()
{
    Animal *animal = new Cat("Tom");
    animal->Speak();

    //通过父类指针去释放，会导致子类对象可能清理不干净，造成内存泄漏
    delete animal;
}

int main()
{
    test();
    return 0;
}
```

文件操作

文件类型分为两种：

1. **文本文件**：文件以文本的**ASCII码**形式存储在计算机中
2. **二进制文件：**文件以文本的**二进制**形式存储在计算机中，用户一般不能直接读懂它们

操作文件的三大类：

1. ofstream：写操作
2. ifstream： 读操作
3. fstream： 读写操作

文本文件

写文件

写文件步骤

1. 包含头文件

   `#include<fstream>`

2. 创建流对象

   `ofsteam ofs`

3. 打开文件

   `ofs.open("文件路径","打开方式");`

4. 写数据

   `ofs << "写入的数据"`

5. 关闭文件

   `ofs.close()`

文件打开方式

打开方式	解释
ios::in	为读文件而打开文件
ios::out	为写文件而打开文件
ios::ate	初始位置：文件尾
ios::app	追加方式写文件
ios::trunc	如果文件存在先删除，再创建
ios::binary	二进制方式

注意：文件打开方式可以配合使用，利用 | 操作符

例如：用二进制方式写文件`ios::binary | ios::out`

注意：

- 文件操作必须包含头文件fstream
- 读文件可以利用ofstream，或者fstream类
- 打开文件时候需要指定操作文件的路径，以打开方式

- 利用 << 可以向文件中写数据
- 操作完毕后需要关闭文件

```
#include <fstream>

void test()
{
    ofstream ofs;
    ofs.open("test.txt", ios::out);

    ofs << "姓名：张三" << endl;

    ofs.close();
}

int main()
{
    test();
    return 0;
}
```

读文件

步骤：

1. 包含头文件

   `#include <fstream>`

2. 创建流对象

   `ifstream ifs`

3. 打开文件并判断文件是否打开成功

   `ifs.open("文件路径",打开方式)`

4. 读数据

   四种方式读取

5. 关闭文件

   `ifs.close()`

注意：

- 读文件可以利用ifstream，或者fstream类
- 利用is_open函数可以判断文件是否打开成功
- close关闭文件

```
#include <fstream>
#include <string>

```

```
void test()
{
    ifstream ifs;
    ifs.open("test.txt", ios::in);

    if(!ifs.is_open())
    {
        cout << "文件打开失败" << endl;
        return;
    }

    //第一种方式
    char buf[1024] = {0};
    while(ifs >> buf)
    {
        cout << buf << endl;
    }

    //第二种方式
    char buf[1024] = {0};
    while(ifs.getline(buf, sizeof(buf)))
    {
        cout << buf << endl;
    }

    //第三种方式
    string buf;
    while(getline(ifs, buf))
    {
        cout << buf << endl;
    }

    //第四种方式
    char c;
    while((c = ifs.get()) != EOF) //EOF : error of file
    {
        cout << c;
    }

    ofs.close();
}

int main()
{
    test();
    return 0;
}
```

二进制文件

以二进制的方式对文件进行读写操作

打开方式要指定为`ios :: binary`

写文件

二进制方式写文件主要利用流对象调用成员函数write

函数原型：`ostream &write(const char* buffer, int len)`

参数解释：字符指针buffer指向内存中一段存储空间，len是读写的字节数

```
class Person
{
    public:
    char m_N[64];
    int m_Age;
}

void test()
{
    ofstream ofs;

    ofs.open("person.txt", ios::out | ios::binary);

    //以上两步可以合成为一步：ofstream ofs("person.txt", ios::out | ios::binary);

    Person p = {"张三", 18};
    ofs,write((const char*)&p, sizeof(Person));

    ofs.close();
}

int main()
{
    test();
    return 0;
}
```

读文件

二进制方式读文件主要利用流对象调用成员函数read

函数原型：`istream& read(char *buffer, int len)`

参数解释：字符指针buffer指向内存中一段存储空间，len是读写的字节数

```cpp
class Person
{
    public:
    char m_N[64];
    int m_Age;
}

void test()
{
    ifstream ifs;

    ifs.open("person.txt", ios::in | ios:: binary);

    if(!ifs.is_open())
    {
        cout << "文件打开失败" << endl;
        return;
    }

    Person p;
    ifs.read((char *)&p, sizeof(Person));
    cout << "姓名: " << p.m_Name << "年龄: " << p.m_Age << endl;

    ifs.close();
}

int main()
{
    test();
    return 0;
}
```

模板

c++ 的编程思想也叫泛型编程，主要利用的技术就是模板

C++提供两种模板机制：**函数模板**和**类模板**

模板的目的是为了提高复用率，将类型参数化

函数模板

介绍

函数模板作用：建立一个通用函数，其函数返回值类型和形参类型可以不具体制定，用一个**虚拟的类型**来代表

语法：

```
template<typename T>
函数声明或定义
```

解释：

`template：声明创建模板`

`typename：表明其后面的符号是一种数据类型，可以用class代替`

`T：通用的数据类型，名称可以替换，通常为大写字母`

使用函数模板有两种方式

- 自动类型推导：让编译器自己推出数据类型
- 显示指定类型：给编译器指定数据类型

注意：

- 自动类型推导，必须推导出一致的数据类型T，才可以使用
- 模板必须要确定出T的数据类型，才可以使用（声明模板后，在声明下方距离声明最近的就默认是函数模板）

  即如果模板函数中没有形参，就不能使用自动类型推导，必须得用显示指定类型，给T提供一个数据类型

```
void swapInt(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

void swapDouble(double &c, double &d)
{
    int temp = c;
    c = d;
    d = temp;
}

//以上的两个函数，想要实现的功能都是相同的，但是因为传入的数据类型不一样就需要重写一个函数，太繁杂
//因此可以用函数模板进行归纳，代替上述写法

template<typename T>
void mySwap(T &a, T &b)
{
    T temp = a;
    a = b;
    b = temp;
}

void test()
{
```

```
    // 常规函数交换
    int a = 10;
    int b = 20;
    swapInt(a, b);
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    //模板交换
    //自动类型推导
    mySwap(a, b);
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    //显示指定类型
    mySwap<int>(a, b);
    cout << "a = " << a << endl;
    cout << "b = " << b << endl;
}

int main()
{
    test();
    return 0;
}
```

普通函数与函数模板的区别

- 普通函数调用时可以发生自动类型转换（隐式类型转换）
- 函数模板调用时
  - 利用自动类型推导，**不会**发生隐式类型转换
  - 利用显示指定类型，**可以**发生隐式类型转换

```
//普通函数
int myAdd01(int a, int b)
{
    return a + b;
}

//函数模板
template<typename T>
T myAdd02(T a, T b)
{
    return a + b;
}

void test()
{
    int a = 10;
    int b = 20;
    char c = 'a';
    //普通函数
    cout << myAdd01(a, b) << endl;  //30
```

```
    cout << myAdd01(a, c) << endl;  //109,将char型隐式类型转换为int型对应的ASCII码

    //函数模板
    //cout << myAdd02(a, c) << endl; //error:自动类型推导不会发生隐式类型转换
    cout << myAdd02<int>(a, c) << endl; //109
}

```

普通函数和函数模板的调用规则（编译器自己的）

- 如果函数模板和普通函数都可以调用，优先调用普通函数（普通情况下，不发生隐式类型转换等）
  就算普通函数只有声明，没有实现，会报错，编译器也会优先调用普通函数
- 可以通过空模板参数列表强制调用函数模板
- 函数模板可以发送函数重载
- 如果函数模板可以产生更好的匹配，优先调用函数模板

```
void myPrint(int a, int b)
{
    cout << "调用的普通函数" << endl;
}

template<typename T>
void myPrint(T a, T b)
{
    cout << "调用的函数模板" << endl;
}

//也可以将参数增加一位，形成函数重载
template<typename T>
void myPrint(T a, T b, T c)
{
    cout << "调用的重载函数模板" << endl;
}

void test()
{
    int a = 10;
    int b = 10;
    myPrint(a, b);  //调用的普通函数

    //通过空模板参数列表，强制调用函数模板
    myPrint<>(a, b);    //调用的函数模板

    //重载
    myPrint(a, b, 100); //调用的重载函数模板

    char c1 = 'a';
    char c2 = 'b';

    myPrint(c1, c2);    //调用的函数模板
```

```
}

int main()
{
    test();
    return 0;
}
```

模板的局限性

模板不是万能的，对于部分的自定义数据类型，无法使用模板

解决方案：将函数模板进行重载，利用具体化的模板，可以解决自定义类型的通用化

注意：函数模板的重载需要加关键词

```
class Person
{
    public:
    int mAge;
    string mName;

    Person(int age, string name)
    {
        mAge = age;
        mName = name;
    }
}

template<typename T>
bool myCompare(T &a, T &b)
{
    if(a == b)
    {
        return true;
    }
    else
        return false;
}

template<> bool myCompare(Person &p1, Person &p2)
{
    if(p1.mAge == p2.mAge && p1.mName == p2.mName)
        return true;
    else
        return false;
}

void test()
{
    Person p1(18, "zhangsan");
    Person p2(20, "lisi");

    myCompare(p1, p2);
```

```
}

int main()
{
    test();
    return 0;
}
```

类模板

介绍

类模板作用：建立一个通用类，类中的成员数据类型可以不具体制定，用一个**虚拟的类型**来代表

语法：

```
template<typename T>
类
```

解释：

`template：声明创建模板`

`typename：表明其后面的符号是一种数据类型，可以用class代替`

`T：通用的数据类型，名称可以替换，通常为大写字母`

注意：

- 类模板没有自动类型推导的使用方式
- 类模板在模板参数列表中可以默认参数（如果声明了默认参数，那么下面调用时就不用传入数据类型）

```
template<class NameType, class AgeType = int>  //模板参数列表中的默认参数
class Person
{
    public:
    NameType m_Name;
    AgeType m_Age;

    Person(NameType name, AgeType age)
    {
        m_Name = name;
        m_Age = age;
    }

    void showPerson()
    {
        cout << "name = " << m_Name << " , age = " << m_Age << endl;
```

```
    }
}

void test()
{
    Person<string, int> p1("孙悟空", 999);
    p1.showPerson();

    Person<string> p2("猪八戒", 888);
    p2.showPerson();
}

int main()
{
    test();
    return 0;
}
```

类模板中成员函数创建时机

类模板中成员函数和普通类中成员函数创建时机是有区别的

- 普通类中的成员函数一开始就可以创建
- 类模板中的成员函数在调用时才创建

```
//以下代码直接运行并不报错，因为类模板中的两个函数只在运行时创建
//就算类模板中不知道obj是什么类型，且如果只调用其中一个也不会报错
class Person1
{
    public:
    void showPerson1()
    {
        cout << "Person1 show" << endl;
    }
};

class Person2
{
    public:
    void showPerson2()
    {
        cout << "Person2 show" << endl;
    }
}

template<class T>
class MyClass
{
    public:
    T obj;

```

```
    void fun1()
    {
        obj.showPerson1();
    }

    void fun2()
    {
        obj.showPerson2();
    }
}

int main()
{
    return 0;
}
```

类模板对象做函数参数

一共有三种传入方式：

1. 指定传入类型： 直接显示对象的数据类型
2. 参数模板化：将对象中的参数变为模板进行传递
3. 整个类模板化：将这个对象类型 模板化进行传递

```
template<class NameType, class AgeType = int>  //模板参数列表中的默认参数
class Person
{
    public:
    NameType m_Name;
    AgeType m_Age;

    Person(NameType name, AgeType age)
    {
        m_Name = name;
        m_Age = age;
    }

    void showPerson()
    {
        cout << "name = " << m_Name << " , age = " << m_Age << endl;
    }
}

// 指定传入类型
void printPerson1(Person<string, int> &p)
{
    p.showPerson();
}

void test01()
{
    Person<string, int>p("孙悟空", 100);
```

```
    printPerson1(p);
}

//参数模板化
template<class T1, class T2>
void printPerson2(Person<T1, T2> &p)
{
    p.showPerson();
    cout << "T1的类型为: " << typeid(T1).name() << endl;
}

void test02()
{
    Person<string, int>p("猪八戒", 90);
    printPerson2(p);
}

//整个类模板化
template<class T>
void printPerson3(T &p)
{
    p.showPerson();
    cout << "T的类型为: " << typeid(T).name() << endl;
}

void test03()
{
    Person<string, int>p("唐僧", 50);
    printPerson3(p);
}

int main()
{
    test01();
    test02();
    test03();

    return 0;
}
```

类模板与继承

当类模板碰到继承时，需要注意以下几点

- 当子类继承的父类是一个类模板时，子类在声明的时候，要指定出父类中的T的类型
- 如果不指定，编译器无法给子类分配内存
- 如果想灵活指定出父类中T的类型，子类也需变为类模板

```
template<class T>
class Base
{
public:
```

```
    T m;
};

//声明变量类型
class Son1 : public Base<int>
{

};

//子类写成模板形式
template<class T1, class T2>
class Son2 : public Base<T2>
{
public:
    Son2()
    {
        cout << "T1 的类型" << typeid(T1).name() << endl;
        cout << "T2 的类型" << typeid(T2).name() << endl;
    }
};

void test01()
{
    Son1 c;
}

void test02()
{
    Son2<int, char> s;
}

int main()
{
    test01();
    test02();
    return 0;
}
```

类模板成员函数的类外实现

类模板中成员函数类外实现时，需要加上模板参数列表

变量类型声明写在类名后

```
template<class TypeName, class TypeAge>
class Person
{
public:
    TypeName mName;
    TypeAge mAge;

    Person(string name, int age);

```

```
    void showPerson();
};

template<class TypeName, class TypeAge>
Person<TypeName, TypeAge>::Person(string name, int age)
{
    mName = name;
    mAge = age;
}

template<class TypeName, class TypeAge>
void Person<TypeName, TypeAge>::showPerson()
{
    cout << "姓名: " << mName << endl;
    cout << "年龄: " << mAge << endl;
}

void test()
{
    Person<string, int> p("孙悟空", 888);
    p.showPerson();
}

int main()
{
    test();
    return 0;
}
```

类模板分文件编写

问题来源：类模板中成员函数创建时机是在调用阶段，导致分文件编写时链接不到

解决方法：

- 方案一：直接包含.cpp源文件

```
//头文件person.h
#prama once
#include <iostream>
#include <string>
using namespace std;
template<class TypeName, class TypeAge>
class Person
{
public:
    TypeName mName;
    TypeAge mAge;

    Person(string name, int age);

    void showPerson();
};

```

```

//源文件一：person.cpp
#include "person.h"
template<class TypeName, class TypeAge>
Person<TypeName, TypeAge>::Person(string name, int age)
{
    mName = name;
    mAge = age;
}

template<class TypeName, class TypeAge>
void Person<TypeName, TypeAge>::showPerson()
{
    cout << "姓名：" << mName << endl;
    cout << "年龄：" << mAge << endl;
}

//源文件二：main.cpp

#include "person.cpp"
void test()
{
    Person<string, int> p("孙悟空", 888);
    p.showPerson();
}

int main()
{
    test();
    return 0;
}
```

- 方案二：将声明和实现写到同一个文件中，并更改后缀名为.hpp，hpp是约定的名称，并不是强制

```
//头文件person.h
#prama once
#include <iostream>
#include <string>
using namespace std;
template<class TypeName, class TypeAge>
class Person
{
public:
    TypeName mName;
    TypeAge mAge;

    Person(string name, int age);

    void showPerson();
};

template<class TypeName, class TypeAge>
Person<TypeName, TypeAge>::Person(string name, int age)
```

```
{
    mName = name;
    mAge = age;
}

template<class TypeName, class TypeAge>
void Person<TypeName, TypeAge>::showPerson()
{
    cout << "姓名：" << mName << endl;
    cout << "年龄：" << mAge << endl;
}

//源文件：main.cpp
#include "person.hpp"
void test()
{
    Person<string, int> p("孙悟空", 888);
    p.showPerson();
}

int main()
{
    test();
    return 0;
}
```

类模板与友元

参考上面类模板成员函数的类外实现

全局函数类内实现：直接在类内声明友元即可

全局函数类外实现：需要提前让编译器知道全局函数的存在

```
#include <iostream>
#include <string>

using namespace std;

//类外实现需要声明类，并且在类上面实现，类似于函数声明原理
//如果不想声明类可以加template<class T1, class T2>void printPerson2(Person<T1, T2> &p);
//将函数声明就可以在类下面实现了
template<class T1, class T2> class Person;

template<class T1, class T2>
void printPerson2(Person<T1, T2> &p) {
    cout << "类外实现" << endl;
    cout << "name = " << p.mName << endl;
    cout << "age = " << p.mAge << endl;
}

```

```
template<class T1, class T2>
class Person {
    friend void printPerson1(Person<T1, T2> &p){
        cout << "类内实现" << endl;
        cout << "name = " << p.mName << endl;
        cout << "age = " << p.mAge << endl;
    }

    friend void printPerson2<>(Person<T1, T2> &p);
private:
    T1 mName;
    T2 mAge;


public:
    Person(T1 name, T2 age) {
        mName = name;
        mAge = age;
    }
};

void test() {
    //类内实现:
    Person<string, int> p1("类内", 20 );
    printPerson1(p1);

    //类外实现:
    Person<string, int> p2("类外", 20);
    printPerson2(p2);
}

int main(){
    test();
    return 0;
}

```

STL

STL六大组件：

- 容器：各种数据结构，如vector、list、deque、set、map等，用来存放数据（**每个容器在使用时都需要包含头文件**）
    - 序列式容器：强调值的排序，每个元素都有固定的位置
    - 关联式容器：二叉树结构，各元素之间没有严格的物理上的顺序关系
- 算法（algorithms）：各种常用的算法，如sort、find、copy、for_each等
    - 质变算法：指运算过程中会更改区间内的元素内容，如拷贝、替换
    - 非质变算法：指运算过程中不会更改区间内的元素内容，如查找、计数、遍历
- 迭代器（iterator）：扮演了容器与算法之间的胶合剂，每个容器都有自己专属的迭代器
    - 迭代器使用非常类似于指针

    - **调用迭代器时操作，如：`vector<int> :: iterator it`（it是自定义变量）**
- 仿函数：行为类似函数，可作为算法的某种策略
- 适配器（配接器）：一种用来修饰容器或者仿函数或迭代器接口的东西
- 空间配置器：负责空间的配置与管理

注意：不支持随机访问迭代器的容器，不可以用标准算法，但是其内部会提供对应一些算法

vector

vector与数组的异同

- 同：都是单端数组
- 异：数组是静态空间，vector可以动态扩展

动态扩展：并不是在原空间之后接新空间，而是找更大的内存空间，然后将原数据拷贝进新空间，释放原空间

构造函数

函数原型

- `vector<T> v` //采用模板类实现，默认构造函数
- `vector<v.begin(), v.end()> v` //将[v.begin(), v.end()) 区间中的元素拷贝给本身
- `vector<n, elem> v` //构造函数将 n 个 elem 拷贝给本身
- `vector<const vector &vec> v` //拷贝构造函数

```
void print_vector(vector<int> &v) {

    //v.begin() 起始迭代器，指向容器第一个元素
    //v.end（）  结束迭代器，指向容器最后一个元素的下一个位置
    for(vector<int>::iterator it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void test01() {
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);    //v1[i] = i; 也行
    }
    print_vector(v1);

    vector<int> v2(v1.begin(), v1.end());
    print_vector(v2);

    vector<int> v3(10,100);
    print_vector(v3);

    vector<int> v4(v3);
```

```
    print_vector(v4);

}

int main(){
    test01();
    return 0;
}
```

赋值

函数原型

- `vector& operator=(const vector &vec)` //重载等号操作符
- `assign(beg, end)` //将[beg, end) 区间中的数据拷贝赋值给本身
- `assign(n, elem)` //将n个elem拷贝赋值给本身

```
void print_vector(vector<int> &v) {
    for(vector<int>::iterator it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void test01() {
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_vector(v1);

    vector<int> v2;
    v2 = v1;
    print_vector(v2);

    vector<int> v3;
    v3.assign(v1.begin(),v1.end());
    print_vector(v3);

    vector<int> v4;
    v4.assign(10,100);
    print_vector(v4);
}

int main(){
    test01();
    return 0;
}
```

容量和大小

- `empty()` //判断容器是否为空
- `capacity()` //容器的容量
- `size()` //返回容器中元素的个数
- `resize(int num)` //重新指定容器的长度为num，若容器变长，则以默认值0填充新位置
  //如果容器变短，则末尾超出容器长度的元素被删除
- `resize(int num, elem)` //重新指定容器的长度为num，若容器变长，则以elem值填充新位置
  //如果容器变短，则末尾超出容器长度的元素被删除

```
void print_vector(vector<int> &v) {

    if(v.empty()) {
        cout << "vector为空！ " << endl;
    }
    else {
        for(vector<int>::iterator it = v.begin(); it != v.end(); it++) {
            cout << *it << " ";
        }
        cout << endl;
        cout << "vector 不为空！ " << endl;
        cout << "vector 的capacity" << v.capacity() << endl;
        cout << "vector 的size " << v.size() << endl;
    }

}

void test01() {
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_vector(v1);

    v1.resize(15);
    print_vector(v1);

    v1.resize(15,100);
    print_vector(v1);

    v1.resize(5);
    print_vector(v1);

}

int main(){
    test01();
    return 0;
}
```

插入和删除

原型

- `push_back(ele)` //尾部插入元素ele
- `pop_back()` //删除最后一个元素
- `insert(const_iterator pos, ele)` //迭代器指向pos插入元素ele
- `insert(const iterator pos, int count, ele)` //迭代器指向pos插入count个元素ele
- `erase(const_iterator pos)` //删除迭代器指向的元素
- `erase(const_iterator start, const_iterator end)` //删除迭代器从start到end之间的元素
- `clear()` //删除容器中所有元素

```
void print_vector(vector<int> &v) {

    if(v.empty()) {
        cout << "vector为空！ " << endl;
    }
    else {
        for(vector<int>::iterator it = v.begin(); it != v.end(); it++) {
            cout << *it << " ";
        }
        cout << endl;
        cout << "vector 不为空！ " << endl;
    }

}

void test01() {
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_vector(v1);

    v1.pop_back();
    print_vector(v1);

    v1.insert(v1.begin(),100);
    print_vector(v1);

    v1.insert(v1.begin(),3,1000);
    print_vector(v1);

    v1.erase(v1.begin());
    print_vector(v1);

    v1.erase(v1.begin(),v1.end());
    print_vector(v1);


    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
```

```
    }
    v1.clear();
    print_vector(v1);
}

int main(){
    test01();
    return 0;
}

```

数据存入

函数原型

- `at(int idx)` //返回索引 idx 所指的数据
- `operator[]` //返回索引 idx 所指的数据
- `front()` //返回容器中的第一个数据元素
- `back()` //返回容器中的最后一个数据元素

```
void test01() {
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }

    cout << "[]访问" << endl;
    for(int i = 0; i <= 10; i++) {
        cout << v1[i] << " ";
    }
    cout << endl;

    cout << "at访问" << endl;
    for(int i = 10; i >= 0; i--) {
        cout << v1.at(i) << " ";
    }
    cout << endl;

    cout << "v1 的第一个元素: " << v1.front() << endl;

    cout << "v1 的最后一个元素: " << v1.back() << endl;


}

int main(){
    test01();
    return 0;
}

```

互换容器

函数原型

- `swap(vec)`　　//将vec与本身元素互换

注意：巧用swap可以收缩内存

```
void print_vector(vector<int> &v) {

    if(v.empty()) {
        cout << "vector为空！ " << endl;
    }
    else {
        for(vector<int>::iterator it = v.begin(); it != v.end(); it++) {
            cout << *it << " ";
        }
        cout << endl;
        cout << "vector 不为空！ " << endl;
    }

}

void test01() {
    cout << "交换前" <<endl;
    vector<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_vector(v1);

    vector<int> v2;
    for(int i = 10; i >= 0; i--) {
        v2.push_back(i);
    }
    print_vector(v2);

    v1.swap(v2);

    cout << "交换后" << endl;
    print_vector(v1);
    print_vector(v2);
}

//巧用swap（）
void print_vector2(vector<int> &v) {

    cout << "v的容量capacity为" << v.capacity() << endl;
    cout << "v的大小size为" << v.size() << endl;
    cout << "----------------" << endl;
}

void test02() {
    vector<int> v;
```

```
    for(int i = 0; i <= 10000; i++) {
        v.push_back(i);
    }
    print_vector2(v);

    v.resize(3);
    print_vector2(v);

    //此处代码行的解释：
    //将代码行拆分为 vector<int> (v) 和 .swap(v)
    //vector<int> (v) 实际上是一个匿名对象，并将v的值传入给此匿名对象
    //.swap(v) 是将v与swap进行交换
    //此句话需要一步完成，不可拆分写，因为匿名对象此句运行完后就自动释放，从而达到收缩内存的作用
    vector<int>(v).swap(v);
    print_vector2(v);
}

int main(){
    test02();
    return 0;
}

```

预留空间

函数原型

- `reserve(int len)` //容器预留 len 个元素长度，预留位置不初始化，元素不可访问

```
void test01() {

    int* p1 = NULL;
    int num1 = 0;
    vector<int> v;
    for(int i = 0; i <= 1000; i++) {
        v.push_back(i);
        if(p1 != &v[0]) {
            p1 = &v[0];
            num1++;
        }
    }
    cout << "开辟空间次数：" << num1 << endl;
    //p1所指向的首地址因为每次扩容会改变的原因：
    //因为vector扩容操作并不是在原来的基础上在后面追加扩容
    //而是找一块新的内存，将原来的数据粘贴到新内存中，因此地址会改变

    vector<int> v2;
    v2.reserve(10000);
    int *p2 = NULL;
    int num2 = 0;
    for(int i = 0; i <= 1000; i++) {
        v2.push_back(i);
```

```
        if(p2 != &v2[0]) {
            p2 = &v2[0];
            num2++;
        }
    }
    cout << "开辟空间次数: " << num2 << endl;
}


int main(){
    test01();
    return 0;
}

```

存放自定义数据类型

```
class Person {
public:
    string mName;
    int mAge;

    Person(string name, int age) {
        mName = name;
        mAge = age;
    }
};

void test01() {

    vector<Person> v1;
    Person p1("张三", 20);
    Person p2("李四", 20);
    Person p3("王五", 20);
    Person p4("赵六", 20);

    v1.push_back(p1);
    v1.push_back(p2);
    v1.push_back(p3);
    v1.push_back(p4);


    for(vector<Person>::iterator it = v1.begin(); it != v1.end(); it++) {
        cout << "姓名:  " << (*it).mName << " 年龄为: " << (*it).mAge << endl;
    }

}


int main(){
    test01();
    return 0;
```

```
}
```

容器中嵌入容器

相当于多维数组

```
void test01() {

    vector< vector<int> >v;
    vector<int> v1;
    vector<int> v2;
    vector<int> v3;
    vector<int> v4;

    for(int i = 0; i <=4; i++) {
        v1.push_back(i);
        v2.push_back(i+1);
        v3.push_back(i+2);
        v4.push_back(i+3);
    }

    v.push_back(v1);
    v.push_back(v2);
    v.push_back(v3);
    v.push_back(v4);

    for(vector< vector<int> > :: iterator it = v.begin(); it != v.end(); it++) {
        //*it表示的是一个vector类型
        for(vector<int> :: iterator vit = (*it).begin(); vit != (*it).end(); vit++) {
            cout << *vit << " ";
        }
        cout << endl;
    }
}


int main(){
    test01();
    return 0;
}
```

string

string 和 char* 的区别

- char*是一个指针，是 c 语言中的字符串
- string是一个类，类内部封装了`char*`，管理这个字符串，是一个`char*`型的容器

构造函数

原型

- `string()` //创建一个空字符串，例如string str

  `string(const char*s)` //使用字符串s初始化
- `string(const string& str)` //使用一个string对象初始化另一个string对象
- `string(int n, char c)` //使用n个字符c 初始化

```
void test() {
    string s1;

    const char* str = "hello world";

    string s2(str);
    cout << "s2 = " << s2 << endl;
    //hello world

    string s3(s2);
    cout << "s3 = " << s3 << endl;
    //hello world

    string s4(10,'a');
    cout << "s4 = " << s4 << endl;
    //aaaaaaaaaa
}

int main(){
    test();
    return 0;
}
```

赋值

原型：前三种是运算符重载

- `string& operator=(const char*s)` //char*类型字符串 赋值给当前的字符串
- `string& operator=(const srting &s)` //把字符串s赋给当前的字符串
- `string& operator=(char c)` //字符赋值给当前的字符串
- `string& assign(const char*s)` //把字符串s赋给当前的字符串
- `string& assign(const char*s, int n)` //把字符串s的前n个字符赋给当前的字符串
- `string& assign(const string &s)` //把字符串s赋给当前字符串
- `string& assign(int n, char c)` //用 n 个字符c 赋给当前字符串

```
void test() {
    string s1;
    s1 = "hello world";
```

```
    cout << s1 << endl;
    //hello world

    string s2;
    s2 = s1;
    cout << s2 << endl;
    //hello world

    s1 = 'c';
    cout << s1 << endl;
    //c

    string s3;
    s3.assign("use assign");
    cout << s3 << endl;
    //use assign

    s3.assign("use assign", 3);
    cout << s3 << endl;
    //use

    s3.assign(s2);
    cout << s3 << endl;
    //hello world

    s3.assign(10,'c');
    cout << s3 << endl;
    //cccccccccc
}

int main(){
    test();
    return 0;
}

```

拼接

原型：前三个是运算符重载

- `string& operator+=(const char* str)` //重载+=操作符
- `string& operator+=(const char c)` //重载+=操作符
- `string& operator+=(const string &str)` //重载+=操作符
- `string& append(const char*s)` //把字符串 s 连接到当前字符串结尾
- `string& append(const char*s, int n)` //把字符串s 的前 n 个字符连接到当前字符串结尾
- `string& append(const string &s)` //同string& operator+=(const string &str)
- `string& append(const string &s, int pos, int n)` //字符串 s 中从 pos 开始的n个字符连接到字符串结尾

```
void test() {
```

```
    string s1;
    s1 = "我";
    cout << s1 << endl;
    //我

    s1 += "玩游";
    cout << s1 << endl;
    //我玩游

    s1 += '戏';
    cout << s1 << endl;
    //我玩游戏

    string s2 = "LOL DNF";
    s1 += s2;
    cout << s1 << endl;
    //我玩游戏LOL DNF

    string s3 = "I";
    s3.append(" love ");
    cout << s3 << endl;
    //I love

    s3.append("game aaaa", 4);
    cout << s3 << endl;
    //I love game

    s3.append(s2,4,3);
    cout << s3 << endl;
    //I love gameDNF

}

int main(){
    test();
    return 0;
}
```

查找和替换

原型：find从左往右查找，rfind从右往左查找

- `int find(const string& str, int pos = 0) const` //查找str第一次出现的位置，从pos开始查找
- `int find(const char* s, int pos = 0) const` //查找s第一次出现的位置，从pos开始查找
- `int find(const char* s, int pos, int n) const` //从pos位置查找s的前n个字符第一次位置
- `int find(const char c, int pos = 0) const` //查找字符c 第一次出现位置
- `int rfind(const string& str, int pos = npos) const` //查找str最后一次位置，从pos开始查找

- `int rfind(const char* s, int pos = npos) const` //查找str最后一次位置，从pos开始查找
- `int rfind(const char* s, int pos, int n) const` //从pos位置查找s的前n个字符最后一次位置
- `int rfind(const char c, int pos = 0) const` //查找字符c 最后一次出现位置
- `string& replace(int pos, int n, const string& str)` //替换从pos开始n个字符为字符串str
- `string& replace(int pos, int n, const char *s)` //替换从pos开始的n个字符为字符串s

```
//查找
void test01() {
    string s1 = "abcbc";
    int pos = s1.find("bc");
    cout << "find pos = " << pos << endl;
    if(pos == -1) {
        cout << "未找到字符串" << endl;
    }
    //find pos = 1


    //rfind
    pos = s1.rfind("bc");
    cout << "rfind pos = " << pos << endl;
    //rfind pos = 3
}

//替换
void test02() {
    string s1 = "abcde";
    s1.replace(1,3, "1111");
    cout << s1 << endl;
    //a1111e
}

int main(){
    test01();
    test02();
    return 0;
}
```

字符串比较

比较方式：字符串按字符的ASCII码进行对比

- = 返回 0
- `>` 返回 1
- < 返回 -1

原型

- `int compare(const string &s) const`
- `int compare(const char *s) const`

```
void test01() {
    string s1 = "xello";
    string s2 = "hello";
    if(s1.compare(s2) == 0) cout << "相等 " << endl;

    if(s1.compare(s2) > 0) cout << "s1 > s2 " << endl;

    if(s1.compare(s2) < 0) cout << "s1 < s2 " << endl;
}



int main(){
    test01();
    return 0;
}

```

字符存取

原型：

- `char& operator[](int n)` //通过[]方式取字符
- `char& at(int n)` //通过at方式取字符

```
void test01() {
    string s1 = "hello";

    //[]的方式
    for(int i = 0; i < s1.size(); i++)  cout << s1[i] << " ";
    //h e l l o

    cout << endl;
    //at的方式
    for(int i = 0; i < s1.size(); i++)  cout << s1.at(i) << " ";
    //h e l l o
}

int main(){
    test01();
    return 0;
}
```

string 插入和删除

原型

- `string& insert(int pos, const char* s)` //插入字符串
- `string& insert(int pos, const string& str)` //插入字符串
- `string& insert(int pos, int n, char c)` //在指定位置插入n个字符c
- `string& erase(int pos, int n = npos)` //删除从pos开始的n个字符

```
void test01() {
    string s1 = "hello";
    s1.insert(1, "111");
    cout << s1 << endl;
    //h111ello

    s1.erase(1,3);
    cout << s1 << endl;
    //hello
}



int main(){
    test01();
    return 0;
}
```

截取子串

原型

- `string substr(int pos = 0, int n = npos) const` //返回由pos开始的n个字符组成的字符串

```
void test01() {
    string s1 = "hello";
    string subStr = s1.substr(1,3);
    cout << s1 << endl;
    //hello
    cout << subStr << endl;
    //ell

    //配合操作
    string email = "2667084782@qq.com";
    int pos = email.find("@");

    cout << email.substr(0,pos) << endl;
    //2667084782
}


```

```

int main(){
    test01();
    return 0;
}

```

deque

功能：双端数组，可以对头端进行插入删除操作

deque 与 vector 的区别：

- vector 对于头部的插入删除效率低，数据量越大，效率越低
- deque 相对而言，对头部的插入删除速度会比 vector 快
- vector 访问元素时的速度会比 deque 快，这和两者内部实现有关

构造函数

- `deque<T> deqT` //默认构造形式
- `deque<beg, end>` //构造函数将 [beg, end)区间中的元素拷贝给本身
- `deque<n, elem>` //构造函数将n个elem拷贝给本身
- `deque<const deque &deq>` //拷贝构造函数

```
//防止打印中修改数组值，因此加const修饰符，注意迭代器的const有特殊写法
void print_vector(const deque<int> &v) {

    //v.begin() 起始迭代器，指向容器第一个元素
    //v.end（）  结束迭代器，指向容器最后一个元素的下一个位置
    for(deque<int>::const_iterator it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void test01() {
    deque<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);    //v1[i] = i; 也行
    }
    print_vector(v1);

    deque<int> v2(v1.begin(), v1.end());
    print_vector(v2);

    deque<int> v3(10,100);
    print_vector(v3);

```

```
    deque<int> v4(v3);
    print_vector(v4);

}

int main(){
    test01();
    return 0;
}
```

赋值操作

函数原型

- `deque& operator=(const deque &deq)` //重载等号操作符
- `assign(beg, end)` //将 [beg, end)区间中的数据拷贝赋值给本身
- `assign(n, elem)` //将 n 个elem拷贝赋值给本身

```
void print_vector(const deque<int> &v) {

    //v.begin() 起始迭代器，指向容器第一个元素
    //v.end（）  结束迭代器，指向容器最后一个元素的下一个位置
    for(deque<int>::const_iterator it = v.begin(); it != v.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void test01() {
    deque<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_deque(v1);

    deque<int> v2;
    v2 = v1;
    print_deque(v2);

    deque<int> v3;
    v3.assign(v1.begin(),v1.end());
    print_deque(v3);

    deque<int> v4;
    v4.assign(10,100);
    print_deque(v4);
}

int main(){
    test01();
    return 0;
}
```

大小操作

deque中没有容量capacity的概念

函数原型：

- `deque.empty()` //判断容器是否为空
- `deque.size()` //返回容器中元素的个数
- `deque.resize(int num)` //重新指定容器的长度为num，若容器变长，则以默认值0填充新位置

  //如果容器变短，则末尾超出容器长度的元素被删除
- `deque.resize(int num, elem)` //重新指定容器的长度为num，若容器变长，则以elem值填充新位置

  //如果容器变短，则末尾超出容器长度的元素被删除

```
void print_deque(const deque<int> &v) {

    if(v.empty()) {
        cout << "deque为空！ " << endl;
    }
    else {
        for(deque<int>::const_iterator it = v.begin(); it != v.end(); it++) {
            cout << *it << " ";
        }
        cout << endl;
        cout << "deque 不为空！ " << endl;
        cout << "deque 的size " << v.size() << endl;
    }

}

void test01() {
    deque<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }
    print_deque(v1);

    v1.resize(15);
    print_deque(v1);

    v1.resize(15,100);
    print_deque(v1);

    v1.resize(5);
    print_deque(v1);

}

int main(){
```

```
    test01();
    return 0;
}
```

插入和删除

两端插入操作

- `push_back(elem)` //在容器尾部添加一个数据
- `push_front(elem)` //在容器头部添加一个数据
- `pop_back()` //删除容器最后一个数据
- `pop_front` //删除容器第一个数据

指定位置操作

- `insert(pos, elem)` //在 pos 位置插入一个elem元素的拷贝，返回新数据的位置
- `insert(pos,n, elem)` //在 pos 位置插入n个elem元素的拷贝，无返回值
- `insert(pos, beg, end)` //在 pos 位置插入[beg, end) 区间的数据，无返回值
- `clear()` //清空容器的所有数据
- `erase(beg, end)` //删除[beg, end）区间的数据，返回下一个数据的位置
- `erase(pos)` //删除pos位置的数据，返回下一个数据的位置

```
void print_deque(const deque<int> &v) {

    if(v.empty()) {
        cout << "deque为空！ " << endl;
    }
    else {
        for(deque<int>::const_iterator it = v.begin(); it != v.end(); it++) {
            cout << *it << " ";
        }
        cout << endl;
    }

}

void test01() {
    deque<int> d1;

    d1.push_back(10);
    d1.push_back(20);

    d1.push_front(100);
    d1.push_front(200);
    print_deque(d1);
    //200 100 10 20

    d1.pop_back();
    print_deque(d1);
```

```
    //200 100 10

    d1.pop_front();
    print_deque(d1);
    //100 10

    d1.insert(d1.begin(),1000);
    print_deque(d1);
    //1000 100 10

    d1.insert(d1.begin(), 2, 2000);
    print_deque(d1);
    //2000 2000 1000 100 10

    deque<int> d2;
    d2.push_back(1);
    d2.push_back(2);
    d2.push_back(3);

    d1.insert(d1.begin(),d2.begin(),d2.end());
    print_deque(d1);
    //1 2 3 2000 2000 1000 100 10


    deque<int>::iterator it = d1.begin();
    it++;

    d1.erase(it);
    print_deque(d1);
    //1 3 2000 2000 1000 100 10

    //注意：若不把迭代器重新赋值，就会造成悬空迭代器，迭代器指向的元素被删，迭代器不知道指向哪里，会默认指向第一次赋值的位置
    it++;
    d1.erase(it, d1.end());
    print_deque(d1);
    //1

    d1.clear();
    print_deque(d1);

}

int main(){
    test01();
    return 0;
}

```

数据存储

- `at(int idx)` //返回索引 idx 所指的数据
- `operator[]` //返回索引 idx 所指的数据
- `front()` //返回容器中的第一个数据元素
- `back()` //返回容器中的最后一个数据元素

```
void test01() {
    deque<int> v1;
    for(int i = 0; i <= 10; i++) {
        v1.push_back(i);
    }

    cout << "[]访问" << endl;
    for(int i = 0; i <= 10; i++) {
        cout << v1[i] << " ";
    }
    cout << endl;

    cout << "at访问" << endl;
    for(int i = 10; i >= 0; i--) {
        cout << v1.at(i) << " ";
    }
    cout << endl;

    cout << "v1 的第一个元素: " << v1.front() << endl;

    cout << "v1 的最后一个元素: " << v1.back() << endl;


}

int main(){
    test01();
    return 0;
}
```

排序操作

对于支持随机访问的迭代器的容器，都可以利用sort算法直接对其进行排序

算法:

- `sort(iterator beg, iterator end)` //对beg和end区间内元素进行排序

```
void print_deque(const deque<int> &v) {

    if(v.empty()) {
        cout << "deque为空! " << endl;
    }
    else {
```

```
        for(deque<int>::const_iterator it = v.begin(); it != v.end(); it++)
{
            cout << *it << " ";
        }
        cout << endl;
    }

}

void test01() {
    deque<int> d1;

    d1.push_back(10);
    d1.push_back(20);

    d1.push_front(100);
    d1.push_front(200);
    cout << "排序前" << endl;
    print_deque(d1);
    //200 100 10 20

    cout << "排序后" << endl;
    sort(d1.begin(),d1.end());
    print_deque(d1);

}

int main(){
    test01();
    return 0;
}

```

stack

栈容器：先进后出

栈不允许有遍历行为

常用接口

构造函数

- `stack<T> stk` //stack 采用模板类实现，stack对象的默认构造形式
- `stack(const stack &stk)` //拷贝构造函数

赋值操作

- `stack &operator=(const stack &stk)` //重载等号操作符

数据存取

- `push(elem)` //向栈顶添加元素
- `pop()` //向栈顶移除第一个元素
- `top()` //返回栈顶元素

大小操作

- `empty()` //判断堆栈是否为空
- `size()` //返回栈的大小

queue

队列容器：先进先出

只有队头和队尾能被外界访问，因此不允许有遍历行为

常用接口

构造函数

- `queue<T> que` //queue 采用模板类实现，queue对象的默认构造形式
- `queue(const queue &que)` //拷贝构造函数

赋值操作

- `queue &operator=(const queue &que)` //重载等号操作符

数据存取

- `push(elem)` //向队尾添加元素
- `pop()` //向队头移除第一个元素
- `front()` //返回第一个元素
- `back()` //返回最后一个元素

大小操作

- `empty()` //判断堆栈是否为空
- `size()` //返回栈的大小

```
class Person {
public:
    string name;
    int age;
```

```

    Person(string name, int age) {
        this->name = name;
        this->age = age;
    }
};

void test01() {

    Person p1("孙悟空",8888);
    Person p2("猪八戒",6666);
    Person p3("沙和尚",7777);
    Person p4("唐三藏",100);


    queue<Person> q;
    q.push(p1);
    q.push(p2);
    q.push(p3);
    q.push(p4);
    cout << "出队前，队列的大小" << q.size() << endl;

    while(!q.empty()) {
        cout << "队头元素 姓名为：" << q.front().name << ",年龄为" << q.front().age << endl;
        cout << "队头元素 姓名为：" << q.back().name << ",年龄为" << q.back().age << endl;
        q.pop();
    }

    cout << "出队后，队列的大小" << q.size() << endl;

}

int main(){
    test01();
    return 0;
}

```

list

链表

将数据进行链式存储，STL中的链表是一个双向循环链表

在list中扩容或缩容都不会引起迭代器的失效，在vector等容器中，当数组进行扩容就会引起迭代器的失效，因为数组的扩容是将原来的数组复制重新开辟一块内存，导致地址发送改变

构造函数

原型

- `list<T> lst` //采用模板类实现，默认构造函数
- `list<begin(), end()>` //将[v.begin(), v.end()) 区间中的元素拷贝给本身
- `list<n, elem>` //构造函数将 n 个 elem 拷贝给本身
- `list<const list &lst>` //拷贝构造函数

```
void print_list(list<int> &lst) {

    //v.begin() 起始迭代器，指向容器第一个元素
    //v.end（）  结束迭代器，指向容器最后一个元素的下一个位置
    for(list<int>::iterator it = lst.begin(); it != lst.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

void test01() {
    list<int> lst1;
    for(int i = 0; i <= 10; i++) {
        lst1.push_back(i);
    }
    print_list(lst1);

    list<int> lst2(lst1.begin(), lst1.end());
    print_list(lst2);

    list<int> lst3(10,100);
    print_list(lst3);

    list<int> lst4(lst3);
    print_list(lst4);

}

int main(){
    test01();
    return 0;
}
```

赋值与交换

函数原型

- `list& operator=(const list &vec)` //重载等号操作符
- `assign(beg, end)` //将[beg, end) 区间中的数据拷贝赋值给本身
- `assign(n, elem)` //将n个elem拷贝赋值给本身
- `swap(lst)` //将lst与本身元素互换

用法与vector相同

大小

函数原型

- `empty()` //判断容器是否为空
- `size()` //返回容器中元素的个数
- `resize(int num)` //重新指定容器的长度为num，若容器变长，则以默认值0填充新位置
  //如果容器变短，则末尾超出容器长度的元素被删除
- `resize(int num, elem)` //重新指定容器的长度为num，若容器变长，则以elem值填充新位置
  //如果容器变短，则末尾超出容器长度的元素被删除

用法与deque一样

插入与删除

函数原型

插入操作

- `push_back(elem)` //在容器尾部添加一个数据
- `push_front(elem)` //在容器头部添加一个数据
- `pop_back()` //删除容器最后一个数据
- `pop_front()` //删除容器第一个数据

指定位置操作

- `insert(pos, elem)` //在 pos 位置插入一个elem元素的拷贝，返回新数据的位置
- `insert(pos,n, elem)` //在 pos 位置插入n个elem元素的拷贝，无返回值
- `insert(pos, beg, end)` //在 pos 位置插入[beg, end) 区间的数据，无返回值
- `clear()` //清空容器的所有数据
- `erase(beg, end)` //删除[beg, end）区间的数据，返回下一个数据的位置
- `erase(pos)` //删除pos位置的数据，返回下一个数据的位置
- `remove(elem)` //删除容器中所有与elem值匹配的值

用法出来remove操作，其他都与queue相同

存取

- `front()` //返回容器中的第一个数据元素
- `back()` //返回容器中的最后一个数据元素

没有at和重载【】，因为迭代器不支持随机访问的

其他用法与queue一样

反转和排序

函数原型

- `reverse()` //反转链表
- `sort()` //链表排序

注意：不支持随机访问迭代器的容器，不可以用标准算法，但是其内部会提供对应一些算法，并且需要自己手写算法规则

```
void print_list(list<int> &lst) {

    //v.begin() 起始迭代器，指向容器第一个元素
    //v.end（）  结束迭代器，指向容器最后一个元素的下一个位置
    for(list<int>::iterator it = lst.begin(); it != lst.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}

bool my_compare(int a, int b) {
    return a < b;             //升序
}

void test01() {

    list<int> lst1;
    for(int i = 0; i <= 10; i++) {
        lst1.push_back(i);
    }
    cout << "反转前" << endl;
    print_list(lst1);

    cout << "反转后" << endl;

    lst1.reverse();
    print_list(lst1);

    cout << "排序前" << endl;
    print_list(lst1);

    cout << "排序后" << endl;
    lst1.sort(my_compare);
    print_list(lst1);

}

int main(){
    test01();
    return 0;
}
```

set/multiset

关联式容器，底层结构用二叉树实现。所有元素会在插入时自动排序

set与multiset区别

- set不允许容器中有重复的元素
- multiset允许容器中有重复的元素

构造和赋值

函数原型

构造

- `set<T> st` //默认构造形式
- `set<const set &st>` //拷贝构造函数

赋值

- `set& operator=(const set &st)` //重载等号操作符

用法与其他容器相同

大小和交换

- `empty()` //判断容器是否为空
- `size()` //返回容器中元素的个数
- `swap(st)` //交换两个集合容器

用法与其他容器相同

插入和删除

函数原型

- `insert(elem)` //在容器中插入元素
- `clear()` //清空容器的所有数据
- `erase(beg, end)` //删除[beg, end）区间的数据，返回下一个数据的位置
- `erase(pos)` //删除pos位置的数据，返回下一个数据的位置
- `erase(elem)` //删除容器中所有与elem值匹配的值

用法与其他容器相同，就是将list中的 remove 更换为 erase 了

查找和统计

- `find(key)` //查找key是否存在，若存在，返回该键的元素的迭代器，若不存在返回set.end()
- `count(key)` //统计key的元素个数

set与multiset的区别

因为两者插入数据时有不同，因此两者使用insert（）返回值也不同

```
void test01() {
    set<int> s;
    //set 使用insert插入会返回一个pair值，pair中两个值为迭代器和bool值，迭代器是当前插入位置，bool值是表示是否存在重发数据
    pair<set<int>::iterator, bool> ret = s.insert(10);
    if(ret.second) {
        cout << "第一次插入成功" << endl;
    }

    else {
        cout << "第一次插入失败" << endl;
    }

    ret = s.insert(10);
    if(ret.second) {
        cout << "第二次插入成功" << endl;
    }

    else {
        cout << "第二次插入失败" << endl;
    }

    multiset<int> ms;
    ms.insert(10);
    ms.insert(10);
    for(multiset<int>::iterator it = ms.begin(); it != ms.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;

}


int main(){
    test01();
    return 0;
}
```

内置类型指定排序规则

set默认排序规则是从小到大，利用仿函数，改变排序规则

注意：重载的规则需要加const修饰，保证规则不会受影响改变

```
class MyCompare {
public:
    bool operator()(int a, int b) const{
        return a > b;
    }
};

void test01() {
    set<int> s;
    //set 使用insert插入会返回一个pair值，pair中两个值为迭代器和bool值，迭代器是当前插入位置，bool值是表示是否存在重发数据
    s.insert(40);
    s.insert(20);
    s.insert(10);
    s.insert(30);
    s.insert(50);

    cout << "修改规则前 " << endl;
    for(set<int>::iterator it = s.begin(); it != s.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;

    cout << "修改排序规则后" << endl;

    set<int, MyCompare> s2;
    s2.insert(40);
    s2.insert(20);
    s2.insert(10);
    s2.insert(30);
    s2.insert(50);
    for(set<int, MyCompare>::iterator it = s2.begin(); it != s2.end(); it++) {
        cout << *it << " ";
    }
    cout << endl;
}


int main(){
    test01();
    return 0;
}
```

自定义数据类型指定排序规则

```
class Person {
public:
    string mName;
    int mAge;

    Person(string name, int age) {
        mName = name;
        mAge = age;
    }
};

class MyCompare {
public:
    bool operator()(const Person &p1, const Person &p2) const {
        return p1.mAge > p2.mAge;
    }
};

void test01() {

    set<Person, MyCompare> v1;
    Person p1("张三", 10);
    Person p2("李四", 20);
    Person p3("王五", 50);
    Person p4("赵六", 40);

    v1.insert(p1);
    v1.insert(p2);
    v1.insert(p3);
    v1.insert(p4);


    for(set<Person,MyCompare>::iterator it = v1.begin(); it != v1.end(); it++) {
        cout << "姓名:  " << (*it).mName << " 年龄为: " << (*it).mAge << endl;
    }

}


int main(){
    test01();
    return 0;
}
```

pair

对组，成对出现的数据，利用对组可以返回两个数据

两种创建方式

- `pair<type,type>p(value1. value2)`
- `pair<type, type>p = make_pair(value1, value2)`

map/multimap

map中所有元素都是pair

pair中第一个元素为key（键值），起到索引作用，第二个元素为value（实值）

所有元素都会根据元素的键值自动排序

map/multimap是关联式容器，底层结构是用二叉树实现

map/multimap区别

- map不允许容器有重复key值元素
- multimap允许容器中重复key值元素

构造与赋值

函数原型

构造

- `map<T1, T2> mp` //默认构造形式
- `map<const map &mp>` //拷贝构造函数

赋值

- `map& operator=(const map &mp)` //重载等号操作符

用法与其他容器相同

大小和交换

- `empty()` //判断容器是否为空
- `size()` //返回容器中元素的个数
- `swap(st)` //交换两个集合容器

用法与其他容器相同

插入和删除

函数原型

- `insert(elem)` //在容器中插入元素
- `clear()` //清空容器的所有数据
- `erase(beg, end)` //删除[beg, end）区间的数据，返回下一个数据的位置
- `erase(pos)` //删除pos位置的数据，返回下一个数据的位置
- `erase(key)` //删除容器中值为key的元素

```
void test()
{
    map<int, int> m;

    m.insert(pair<int, int>(1, 10));

    m.insert(make_pair(2, 20));

    m.insert(map<int, int>::value_type(3, 30));

    m[4] = 40;

}
```

查找和统计

- `find(key)` //查找key是否存在，若存在，返回该键的元素的迭代器，若不存在返回set.end()
- `count(key)` //统计key的元素个数

容器排序

利用仿函数，改变排序规则

```
class MyCompare {
public:
    bool operator()(int a, int b) const{
        return a > b;
    }
};

void test01() {

    set<int, int, MyCompare> m;
    m.insert(make_pair(1, 10));
    m.insert(make_pair(1, 20));
    m.insert(make_pair(1, 30));
```

```
    m.insert(make_pair(1, 40));
    m.insert(make_pair(1, 50));

    for(set<int, int, MyCompare>::iterator it = m.begin(); it != m.end(); it++) {
        cout << "key: " << it -> first << " value: " << it -> second << endl;;
    }
}


int main(){
    test01();
    return 0;
}
```

函数对象

重载函数调用操作符类，其对象常称为函数对象

函数对象使用重载的（）时，行为类似函数调用，也叫仿函数

本质：函数对象（仿函数）是一个类，不是一个函数

函数对象的使用

- 仿函数在使用时，可以像普通函数那样调用，可以有参数，可以有返回值
- 函数对象超出普通函数的概念，仿函数可以有直接的状态（普通函数的局部变量运行完后就自动释放了，因此记录不了状态）
- 仿函数可以作为参数传递

谓词

- 返回bool类型的仿函数称为谓词
- 如果operator（）接收一个参数，那么叫做一元谓词
- 如果operator（）接收两个参数，那么叫做二元谓词

内建函数对象

- 算法仿函数
- 关系仿函数
- 逻辑仿函数

头文件：`#include <functional>`

用法：

- 这些仿函数所产生的对象，用法和一般函数完全相同
- 使用内建函数对象，需要引入头文件`#include<functional>`

算法仿函数

- `template<class T> T plus<T>` //加法仿函数
- `template<class T> T minus<T>` //减法仿函数
- `template<class T> T mutiplies<T>` //乘法仿函数
- `template<class T> T divides<T>` //除法仿函数
- `template<class T> T modulus<T>` //取模仿函数
- `template<class T> T negate<T>` //取反仿函数

```
#include <functional>

void test01()
{
    negate<int> n;
    cout << n(50) << endl;
    //-50
}

void test02()
{
    plus<int> p;
    cout << p(10, 20) << endl;
}

int main()
{
    test01();
    test02();
    return 0;
}
```

关系仿函数

函数原型

- `template<class T> bool equal_to<T>` //等于
- `template<class T> bool not_equal_to<T>` //不等于
- `template<class T> bool greater<T>` //大于

- `template<class T> bool greaterequal<T>` //大于等于
- `template<class T> bool less<T>` //小于
- `template<class T> bool less_equal<T>` //小于等于